

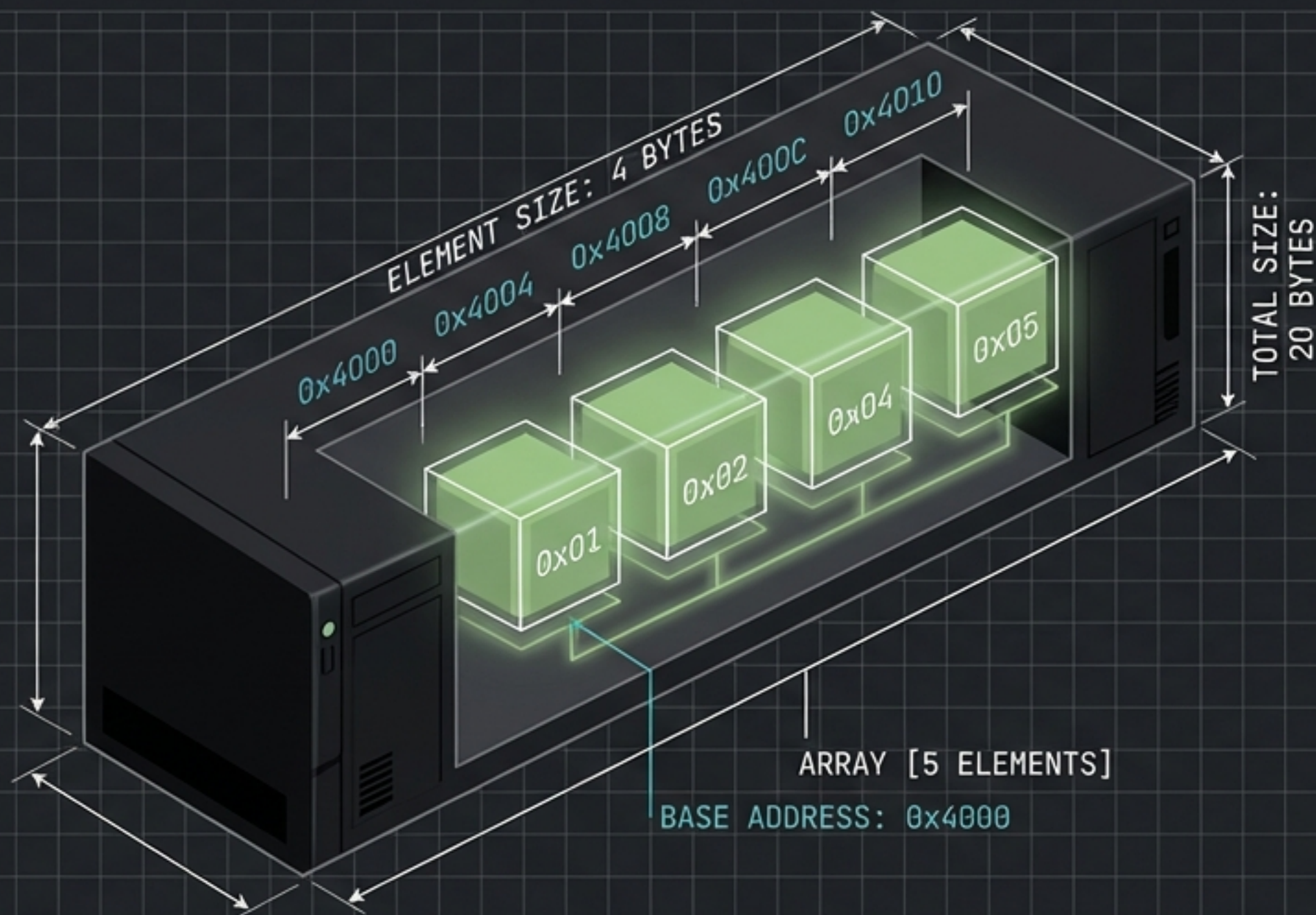
THE ANATOMY OF AN ARRAY

A Forensic Guide to Identifying Data Structures in Assembly

MEMORY LAYOUT (HEX)

ADDRESS	VALUE
0x4000	01 00 00 00
0x4004	02 00 00 00
0x4008	03 00 00 00
0x400C	04 00 00 00
0x4010	05 00 00 00

ARRAY
POINTER → 0x4000



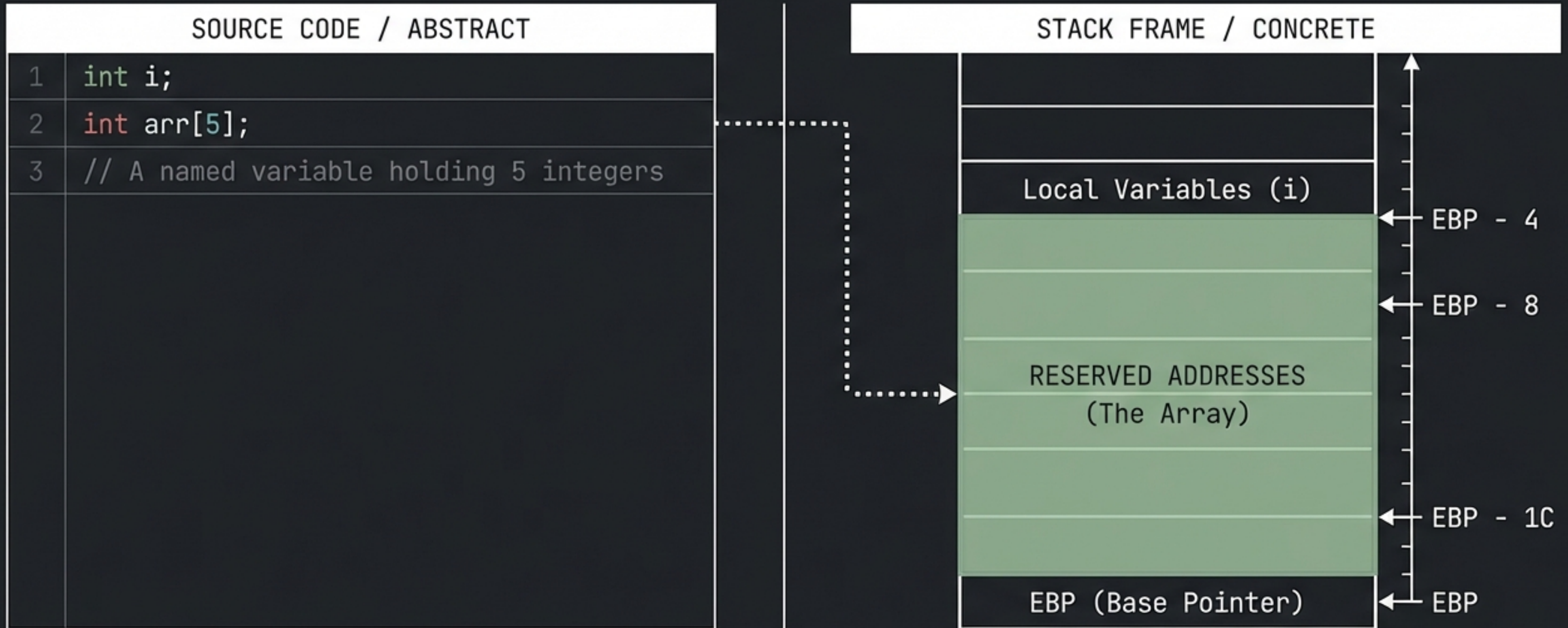
ASSEMBLY INSTRUCTIONS

```
MOV EAX, [0x4000] ; Load array base address into EAX
ADD EAX, 0x1        ; Increment pointer by 1 element (4 bytes)

MOV EBX, [0x4004] ; Load array element at index 1 into EBX
ADD EBX, 0x2       ; Increment value by 2
```

Contiguous memory allocation is a key characteristic of arrays. Each element is accessed via base address + (index * element size).

THE HIGH-LEVEL ILLUSION VS. MEMORY REALITY



In C, it is a named variable. In Assembly, it is simply a contiguous block of reserved stack addresses referenced by their distance from the Base Pointer.

DEFINING THE PERIMETER: CALCULATING ALLOCATION

Element Size (Integer)	_____	4 bytes
Number of Elements	_____	x 5
TOTAL SIZE	_____	20 bytes

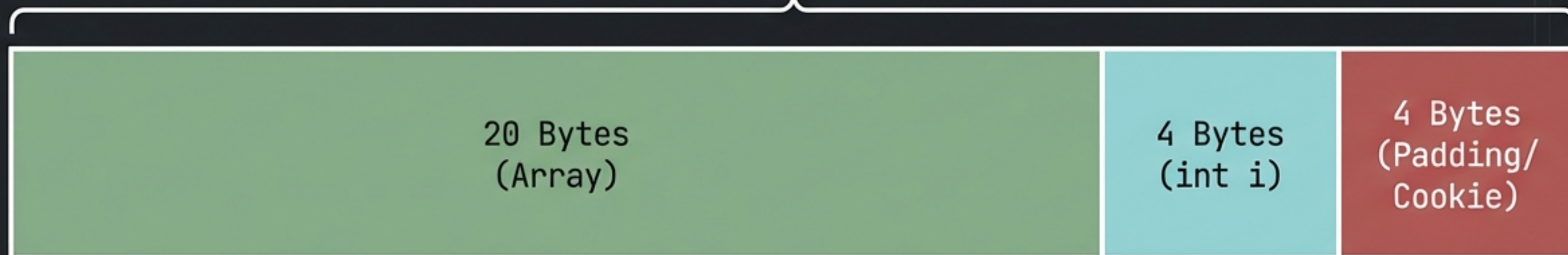
20 (Base 10) $\xrightarrow{\text{Convert to Hexadecimal}}$ **0x14 (Hex)**

To find the array, you must first know its weight. For an integer array of size 5, we are hunting for a 20-byte (0x14) hole in the memory.

THE ALLOCATION SIGNATURE

```
sub esp, 1Ch
```

Total Stack Reservation: 0x1C (28 Bytes)



The instruction 'sub esp, 1Ch' reserves 28 bytes. This accounts for our 20-byte array, our 4-byte loop counter, and the extra security padding. The math aligns.

READING THE GAPS: NEGATIVE SPACE ANALYSIS

var_4 DWORD ? ; Loop Counter

var_18 DWORD ? ; Array Start



$0x18 - 0x4 = 0x14$
 $0x14 \text{ Hex} = 20 \text{ Bytes}$

IDA assigns dummy names like var_18. The array is revealed not by its name, but by the empty space between it and the next variable. A 20-byte gap confirms our 5-integer array.

THE SEQUENTIAL NATURE OF MEMORY

...00416CC4 (Element 0)	} +4 Bytes
...	
...00416CC8 (Element 1)	} +4 Bytes
...	
...00416CCC (Element 2)	} +4 Bytes
...	
...00416CD0 (Element 3)	} +4 Bytes
...	
...00416CD4 (Element 4)	} +4 Bytes
...	

Arrays are contiguous blocks. Unlike linked lists or scattered variables, the addresses here increment in perfect, predictable steps matching the element size (4 bytes).

THE ACCESS MECHANISM: CALCULATING THE STRIDE

$[\text{BASE}] + ([\text{INDEX}] * [\text{SCALE}])$

$\text{mov } \dots, [\text{ebp}-18\text{h} + \text{ecx} * 4]$

The CPU does not know 'index 3'. It only knows to start at the Base Address (var_18) and walk forward 'Index * 4' bytes.

EXECUTION TRACE: INDEX 0

THE MATH

Index (ECX) = 0

Formula: $\text{Base} + (0 * 4)$

Offset result: +0

When the counter is 0, the offset is 0.
We access the Base Address directly.
We access the Base Address directly.

MEMORY MAP



Base Address

EXECUTION TRACE: INDEX 1

THE MATH

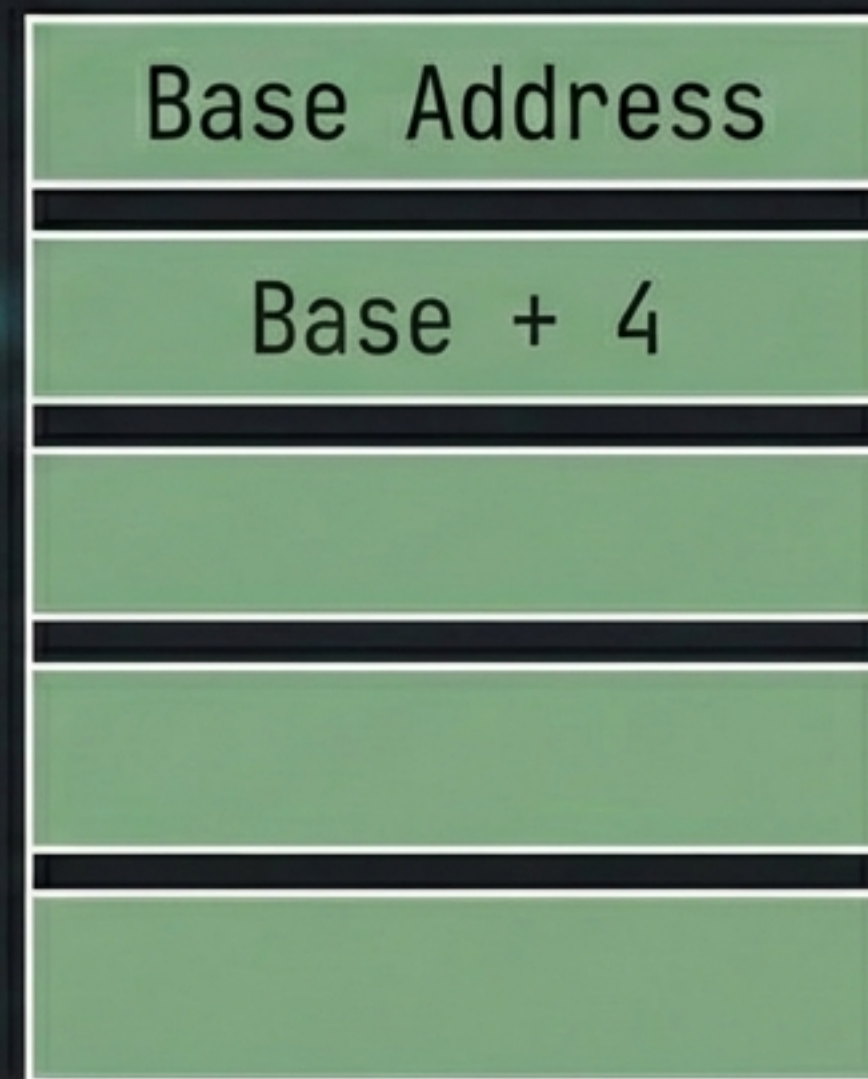
Index (ECX) = 1

Formula: $\text{Base} + (1 * 4)$

Offset result: +4 bytes

When the counter is 1, we add exactly 4 bytes to the Base. We arrive at the second element.

MEMORY MAP



EXECUTION TRACE: INDEX 4 (FINAL)

THE MATH

Index (ECX) = 4

Formula: $\text{Base} + (4 * 4)$

Offset result: +16 bytes (0x10)

MEMORY MAP

Base Address

Base + 4

Base + 8

Base + 12

Base + 0x10

The multiplier drives the stride. Even at the end of the array, the formula holds true, landing us exactly at the final 4-byte block.

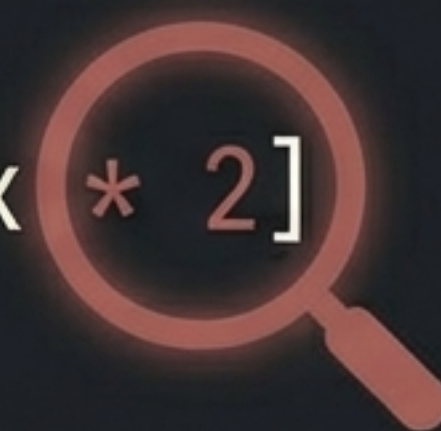
DEDUCTION: THE MULTIPLIER REVEALS THE TYPE

```
lea eax, [ecx * 4]
```



4 Bytes = Integer or Pointer

```
lea eax, [ecx * 2]
```



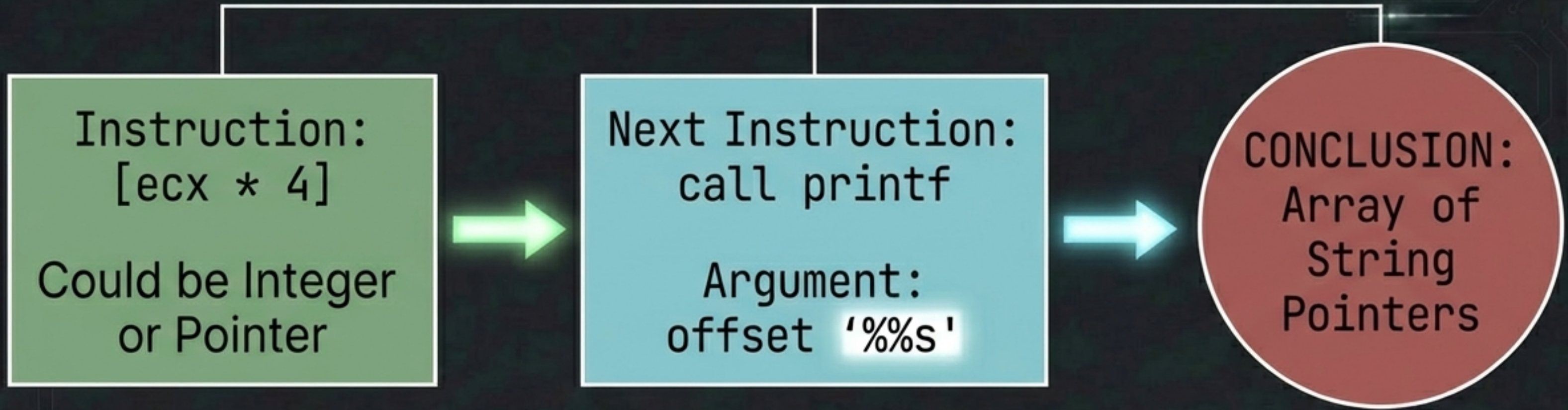
2 Bytes = Short

The scale factor is the DNA of the array.

A multiplier of *4 indicates 32-bit integers or pointers.

A multiplier of *2 indicates 16-bit shorts.

ADVANCED DEDUCTION: CONTEXT IS KEY



Code does not exist in a vacuum. The multiplier tells us the size (4 bytes), but the usage (`printf %s`) proves these are pointers to strings, not raw integers.

THE REVERSE ENGINEER'S CHECKLIST



1. CHECK ALLOCATION

Does “sub esp, X” create enough room for Size * Count?



2. MEASURE THE GAP

Does the distance between local variables (e.g., var_18 to var_4) match the calculated size?



3. INSPECT THE SCALE

In the loop, what is the index multiplied by (*1, *2, *4, *8)?

BEYOND THE ARRAY



We have successfully mapped the array. But what about those extra bytes we identified in the allocation? That anomaly—the Security Cookie—is our next target.

Lecture Source: Code Constructs - Arrays